md-pdf

A comprehensive guide



tschinz - 2026-03-21 - v0.1.3

Tags user-guide documentation md-pdf markdown to pdf typst configuration templates

---

Contents

1 Overview . . . 3
1.1 Key Benefits . . . 3
2 Installation . . . 3
2.1 Prerequisites . . . 3
2.1.1 Rust Toolchain (Optional, for building from source) . . . 3
2.1.2 Typst CLI (Required) . . . 4
2.1.3 Fonts (optional) . . . 4
2.2 Installing md-pdf . . . 4
2.2.1 First Run & Auto-Setup . . . 4
2.2.2 Verification . . . 4
3 Configuration System . . . 5
3.1 Auto-Configuration . . . 5
3.2 Creating Configuration Files . . . 5
3.2.1 Automatic Creation . . . 5
3.2.2 Manual Configuration . . . 5
3.3 Configuration File Locations . . . 5
3.4 Configuration File Content . . . 6
3.5 Configuration Options Reference . . . 6
3.6 Viewing Configuration . . . 6
4 Templates System . . . 7
4.1 Built-in Templates . . . 7
4.1.1 `None` . . . 7
4.1.2 `simple` . . . 7
4.1.3 `playful` . . . 8

4.1.4 **brutalist** 9
4.1.5 **darko** 9
4.2 Template Selection Priority 10
4.3 Template Variables & Data Passing 10
4.3.1 System Variables 10
4.3.2 Front Matter Variables 11
4.3.3 Custom Fields 11
4.4 Using Variables in Templates 11
4.4.1 Basic Variable Access 12
4.4.2 Advanced Variable Processing 12
4.4.3 Variable Validation and Defaults 12
4.5 Custom Template Development 12
4.5.1 Creating Custom Templates 12
5 Front Matter Support 13
5.1 What is Front Matter? 13
5.2 Basic Front Matter 13
5.3 Supported Data Types 13
5.3.1 Strings 13
5.3.2 Numbers 13
5.3.3 Booleans 14
5.3.4 Arrays/Lists 14
5.3.5 Objects/Dictionaries 14
5.3.6 Dates and Times 14
5.4 Field Precedence and Smart Defaults 15
5.5 Front Matter Example 15
6 Working with Images 15
6.1 Resizing Images 16
6.2 Aligning Images 17
6.2.1 Left 17
6.2.2 Center 18
6.2.3 Right 18
6.3 Tips 18
7 Usage Guide 19
7.1 Basic Commands 19
7.1.1 Essential Operations 19
7.1.2 Information and Configuration Commands 20
7.1.3 Link Checking and Validation 20
7.1.3.1 Sample Output 20



1 Overview

md-pdf is a powerful, lightweight command-line tool that converts Markdown files into professional PDF documents. Built with Rust and powered by the Typst typesetting system, it provides fast, high-quality document generation.

md-pdf bridges the gap between Markdown's simplicity and PDF's professional presentation. It's designed for developers, writers, researchers, and anyone who needs to create beautiful documents from Markdown source files.



1.1 Key Benefits

- 🚀 **Lightning Fast**: PDF generated with Typst, minimal installation footprint
- 🎨 **Professional Output**: High-quality typography and layout
- ⚙️ **Zero Configuration**: Works out of the box with intelligent defaults
- 🔧 **Highly Customizable**: Flexible template system and configuration options
- 👀 **Live Preview**: Real-time PDF updates with watch mode
- 📂 **Auto-Open**: Cross-platform PDF viewing with system default applications
- 🔗 **Link Validation**: Built-in checking for external links to ensure document quality
- 📝 **Rich Metadata**: Comprehensive front matter support

2 Installation

2.1 Prerequisites

Before installing md-pdf, you need:

2.1.1 Rust Toolchain (Optional, for building from source)

If you plan to build from source:

```
# Install rustup if needed
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh

# Restart shell or source the environment
source $HOME/.cargo/env

# Verify installation
```




```
rustc --version
cargo --version
```

Visit rust-lang.org for more details.

2.1.2 Typst CLI (Required)

Typst is the typesetting engine that powers PDF generation:

```
# Install via cargo (recommended)
cargo install typst-cli

# Verify installation
typst --version
```

Visit typst.app for more details.

2.1.3 Fonts (optional)

Some templates using custom fonts. If they are not installed a fallback will be used.

- Fira Sans and Fira Mono
- Iosevka

2.2 Installing md-pdf

Install directly from the Git repository using cargo:

```
# Install latest stable release
cargo install --git https://github.com/tschinz/md-pdf

# Install specific version/tag
cargo install --git https://github.com/tschinz/md-pdf --tag v0.1.0
```

2.2.1 First Run & Auto-Setup

After installation, md-pdf is ready to use immediately:

```
# Convert your first document (triggers auto-setup)
md-pdf README.md

# Convert and open PDF automatically
md-pdf README.md --open

# Check what was automatically created
md-pdf --show-config

# List available templates
md-pdf --list-templates
```

On first run, md-pdf automatically creates:

- **Configuration file**: **`~/.config/md-pdf/config.ron`**
- **Templates directory**: **`~/.config/md-pdf/templates/`**
- **Template files**: **`none.typ`**, **`simple.typ`**, **`playful.typ`** and **`brutalist.typ`**

2.2.2 Verification

Verify your installation works correctly:

```
# Check version
md-pdf --version
```




```
# Test conversion with a simple document
echo "# Test Document" > test.md
echo "This is a test." >> test.md
md-pdf test.md

# Check if PDF was created
ls -la test.pdf
```

3 Configuration System

md-pdf uses a sophisticated configuration system that provides intelligent defaults while allowing complete customization.

3.1 Auto-Configuration

The tool automatically configures itself on first use:

- Creates a configuration file with sensible defaults
- Sets up a templates directory with proper paths
- Installs basic template files with full content
- No manual setup required!

3.2 Creating Configuration Files

3.2.1 Automatic Creation

Configuration is created automatically on first use, but you can manually create it:

```
# Create default configuration file
md-pdf --create-config
```

This creates **`~/.config/md-pdf/config.ron`** with:

- Proper templates directory path
- Sensible defaults for all options
- Complete template files (none.typ and simple.typ)
- Detailed comments explaining each option

3.2.2 Manual Configuration

You can also create configurations in specific locations:

```
# Project-specific configuration
echo '(default_template: Some("custom"))' > ./md-pdf.ron

# User-specific override
mkdir -p ~/.config/md-pdf
md-pdf --create-config
```

3.3 Configuration File Locations

md-pdf searches for configuration files in this priority order:

1. **`./md-pdf.ron`** - Project-specific configuration (highest priority)
2. **`<binary-dir>/md-pdf.ron`** - Portable installation configuration
3. **`~/md-pdf.ron`** - User home directory



4. **`~/.config/md-pdf.ron`** - XDG standard location
5. **`~/.config/md-pdf/config.ron`** - **Recommended default location**
6. **`~/.config/zas/md-pdf.ron`** - Workflow integration (lowest priority)

3.4 Configuration File Content

The configuration file uses RON (Rust Object Notation) format, which is human-readable and easy to edit:

```
(
    // Path to templates directory (absolute path recommended)
    // This directory contains .typ template files
    templates_dir: Some("/Users/username/.config/md-pdf/templates"),

    // Default template to use when none specified
    // Available options: "none", "simple", or custom template name
    default_template: Some("simple"),

    // Default language for documents
    // Affects typography, hyphenation, and formatting
    default_language: Some("en"),

    // Generate table of contents by default
    // Can be overridden per document via front matter
    default_toc: Some(true),

    // Default author name for documents without explicit author
    // Used when no author specified in front matter
    default_author: Some("Your Name Here"),
)
```

3.5 Configuration Options Reference

Field	Type	Description	Default Value	Examples
`templates_dir`	**`Option<PathBuf>`**	Directory containing template files	**`~/.config/md-pdf/templates`**	Custom template collections
`default_template`	**`Option<String>`**	Template name to use by default	**`"simple"`**	**`"academic"`**, **`"corporate"`**
`default_language`	**`Option<String>`**	Document language code	**`"en"`**	**`"de"`**, **`"fr"`**, **`"es"`**
`default_toc`	**`Option<bool>`**	Include table of contents	**`true`**	Per-document override available
`default_author`	**`Option<String>`**	Default document author	**`"User"`**	Your name or organization

3.6 Viewing Configuration

Check your current configuration:

```
# Show all configuration details
md-pdf --show-config
```

```
# Output example:
# Configuration Information
# ========================
# Templates directory: /Users/username/.config/md-pdf/templates
# Default template: simple
# Default language: en
# Default TOC: true
# Default author: Your Name
```

4 Templates System

The template system controls the visual appearance and layout of your PDF documents. md-pdf includes built-in templates and supports unlimited custom templates.

4.1 Built-in Templates

4.1.1 `None`

Minimal template with basic styling



USS Enterprise Tech Brief

Markdown Elements Guide

Lt. Commander Data - 2024-12-20 - NCC-1701-D

TAGS starfleet technical markdown

PARTICIPANTS Jean-Luc Picard William Riker Data

USS Enterprise Tech Brief

A concise guide to markdown elements aboard the Federation flagship.

Core Systems

Warp Drive

The **warp core** maintains peak efficiency while *dilithium crystals* power our journey. The ~~old conduits~~ have been upgraded.

Systems Status

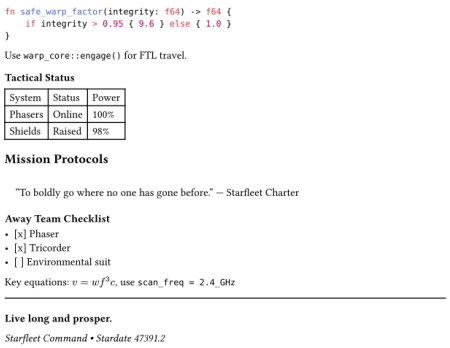
- Warp coils aligned
- Antimatter stable
  - Primary containment
  - Backup protocols

Emergency Steps

1. Initiate shutdown
2. Eject core if needed
3. Switch to impulse

Engineering Code

Ship's computer Rust function:



```rust
fn safe_warp_factor(integrity: f64) -> f64 {
    if integrity > 0.95 { 9.6 } else { 1.0 }
}
```

Use `warp_core::engage()` for FTL travel.

Tactical Status

System	Status	Power
Phasers	Online	100%
Shields	Raised	98%

Mission Protocols

> "To boldly go where no one has gone before." — Starfleet Charter

Away Team Checklist

- [x] Phaser
- [x] Tricorder
- [ ] Environmental suit

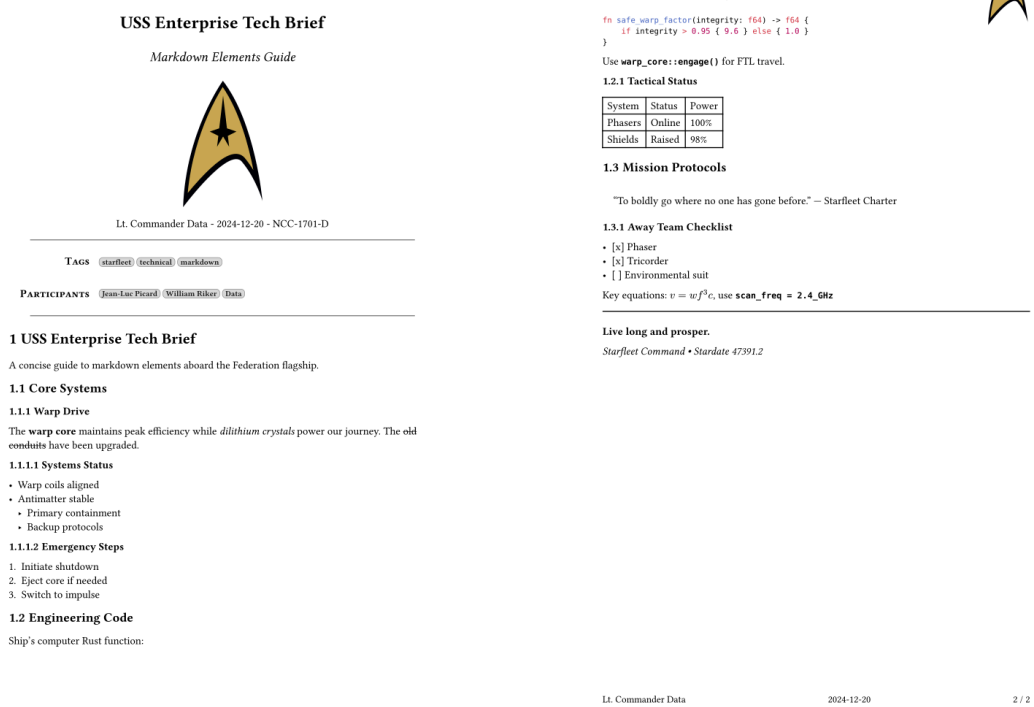
Key equations: $v = wf^3c$, use `scan_freq = 2.4_GHz`

Live long and prosper.

Starfleet Command • Stardate 47391.2

4.1.2 `simple`

Clean template with headers and footers



USS Enterprise Tech Brief

Markdown Elements Guide

Lt. Commander Data - 2024-12-20 - NCC-1701-D

TAGS starfleet technical markdown

PARTICIPANTS Jean-Luc Picard William Riker Data

1 USS Enterprise Tech Brief

A concise guide to markdown elements aboard the Federation flagship.

1.1 Core Systems

1.1.1 Warp Drive

The **warp core** maintains peak efficiency while *dilithium crystals* power our journey. The ~~old~~ ~~conduits~~ have been upgraded.

1.1.1.1 Systems Status

- Warp coils aligned
- Antimatter stable
  - Primary containment
  - Backup protocols

1.1.1.2 Emergency Steps

1. Initiate shutdown
2. Eject core if needed
3. Switch to impulse

1.2 Engineering Code

Ship's computer Rust function:

USS ENTERPRISE TECH BRIEF | MARKDOWN ELEMENTS GUIDE

```rust
fn safe_warp_factor(integrity: f64) -> f64 {
    if integrity > 0.95 { 9.6 } else { 1.0 }
}
```

Use `warp_core::engage()` for FTL travel.

1.2.1 Tactical Status

System	Status	Power
Phasers	Online	100%
Shields	Raised	98%

1.3 Mission Protocols

> "To boldly go where no one has gone before." — Starfleet Charter

1.3.1 Away Team Checklist

- [x] Phaser
- [x] Tricorder
- [ ] Environmental suit

Key equations: $v = wf^3c$, use `scan_freq = 2.4_GHz`

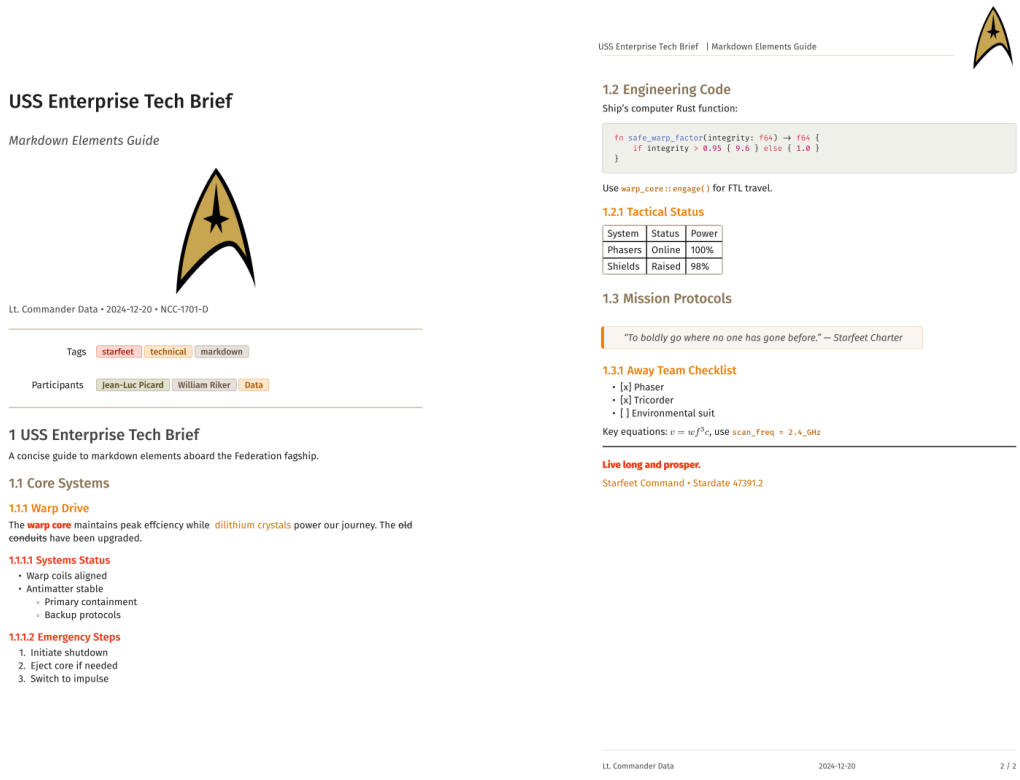
Live long and prosper.

Starfleet Command • Stardate 47391.2

Lt. Commander Data 2024-12-20 2 / 2

4.1.3 `playful`

Colorful Dieter Rams-inspired design



USS Enterprise Tech Brief

Markdown Elements Guide

Lt. Commander Data • 2024-12-20 • NCC-1701-D

Tags starfeet technical markdown

Participants Jean-Luc Picard William Riker Data

1 USS Enterprise Tech Brief

A concise guide to markdown elements aboard the Federation fagship.

1.1 Core Systems

1.1.1 Warp Drive

The **warp core** maintains peak effciency while dilithium crystals power our journey. The ~~old~~ ~~conduits~~ have been upgraded.

1.1.1.1 Systems Status

- Warp coils aligned
- Antimatter stable
  - Primary containment
  - Backup protocols

1.1.1.2 Emergency Steps

1. Initiate shutdown
2. Eject core if needed
3. Switch to impulse

USS Enterprise Tech Brief | Markdown Elements Guide

1.2 Engineering Code

Ship's computer Rust function:

```rust
fn safe_warp_factor(integrity: f64) -> f64 {
    if integrity > 0.95 { 9.6 } else { 1.0 }
}
```

Use `warp_core::engage()` for FTL travel.

1.2.1 Tactical Status

System	Status	Power
Phasers	Online	100%
Shields	Raised	98%

1.3 Mission Protocols

> *"To boldly go where no one has gone before." — Starfeet Charter*

1.3.1 Away Team Checklist

- [x] Phaser
- [x] Tricorder
- [ ] Environmental suit

Key equations: $v = wf^3c$, use `scan_freq = 2.4_GHz`

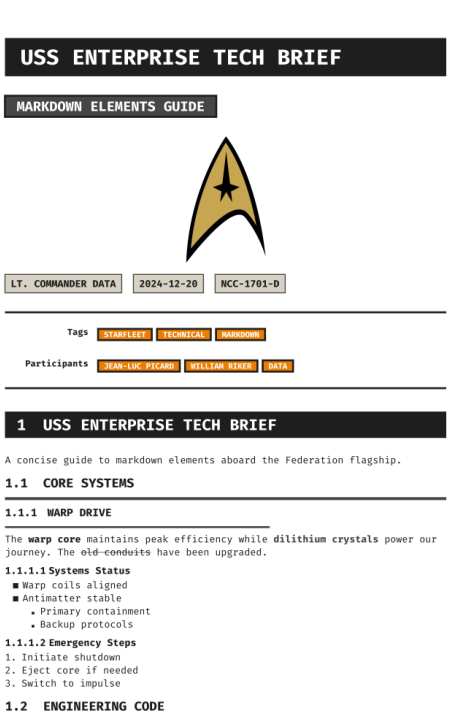
Live long and prosper.

Starfeet Command • Stardate 47391.2

Lt. Commander Data 2024-12-20 2 / 2

4.1.4 `brutalist`

Raw, stark monospace aesthetic



USS ENTERPRISE TECH BRIEF

MARKDOWN ELEMENTS GUIDE

LT. COMMANDER DATA | 2024-12-20 | NCC-1701-D

Tags: STARFLEET | TECHNICAL | MARKDOWN

Participants: JEAN-LUC PICARD | WILLIAM RIKER | DATA

1 USS ENTERPRISE TECH BRIEF

A concise guide to markdown elements aboard the Federation flagship.

1.1 CORE SYSTEMS

1.1.1 WARP DRIVE

The warp core maintains peak efficiency while dilithium crystals power our journey. The ~~old conduits~~ have been upgraded.

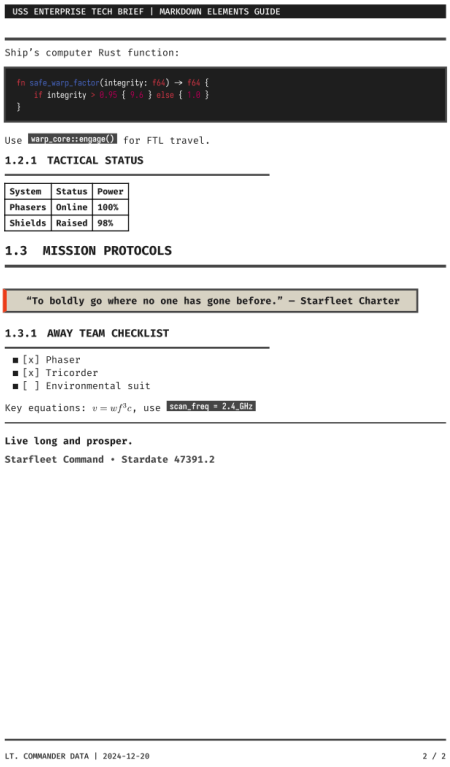
1.1.1.1 Systems Status

- Warp coils aligned
- Antimatter stable
  - Primary containment
  - Backup protocols

1.1.1.2 Emergency Steps

1. Initiate shutdown
2. Eject core if needed
3. Switch to impulse

1.2 ENGINEERING CODE



USS ENTERPRISE TECH BRIEF | MARKDOWN ELEMENTS GUIDE

Ship's computer Rust function:

```
fn safe_warp_factor(integrity: f64) -> f64 {
    if integrity > 0.95 { 9.9 } else { 1.0 }
}
```

Use `warp_core::engage()` for FTL travel.

1.2.1 TACTICAL STATUS

System	Status	Power
Phasers	Online	100%
Shields	Raised	98%

1.3 MISSION PROTOCOLS

> "To boldly go where no one has gone before." — Starfleet Charter

1.3.1 AWAY TEAM CHECKLIST

- [x] Phaser
- [x] Tricorder
- [ ] Environmental suit

Key equations: $v = wf^3 c$, use `scan_freq = 2.4 GHz`

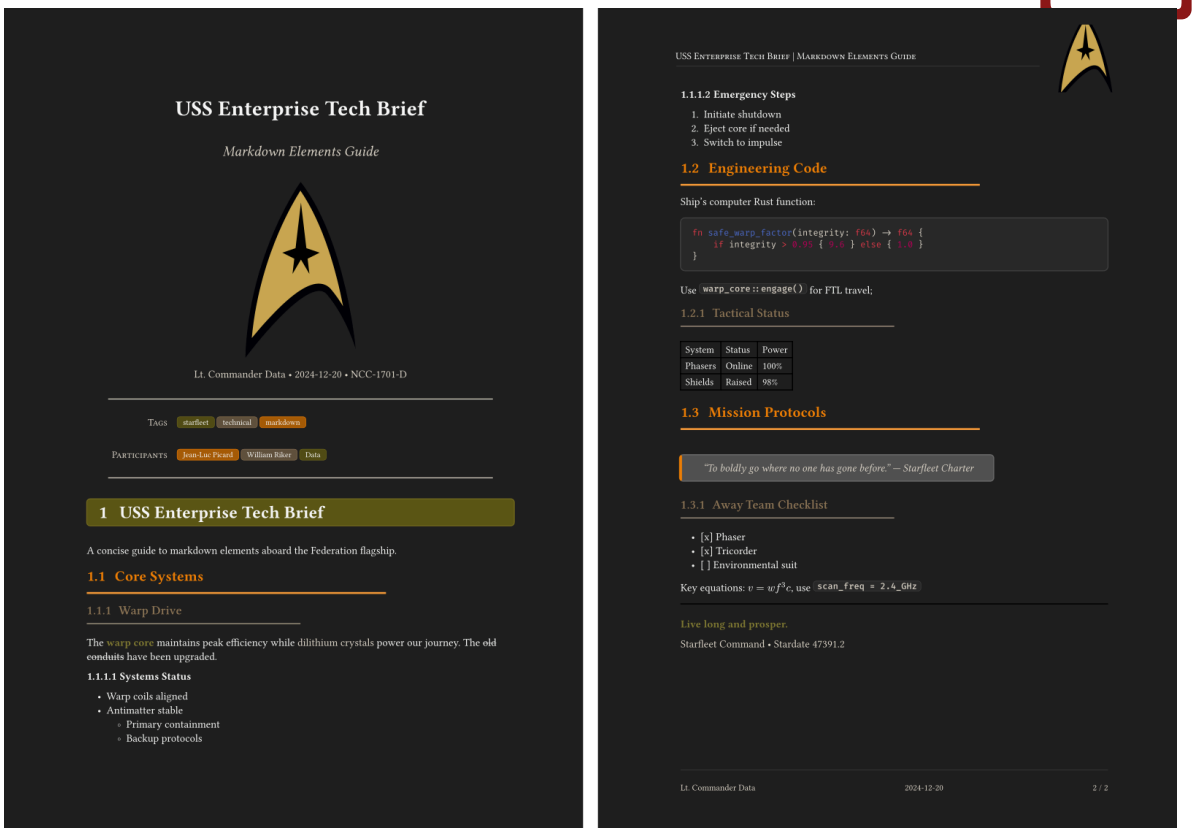
Live long and prosper.

Starfleet Command • Stardate 47391.2

LT. COMMANDER DATA | 2024-12-20 2 / 2

4.1.5 `darko`

May the dark side be with you



4.2 Template Selection Priority

Templates are chosen in this hierarchical order:

1. **Command-line argument**: **`-t template_name`** (highest priority)
2. **Front matter**: **`template: "template_name"`**
3. **Configuration default**: **`default_template`** setting
4. **System fallback**: **`"none"`** template (lowest priority)

Example:

```
# Command-line overrides everything
md-pdf doc.md -t simple  # Uses 'simple' regardless of config/frontmatter
```

4.3 Template Variables & Data Passing

Templates receive comprehensive data from the conversion process. Understanding these variables is crucial for creating custom templates.

4.3.1 System Variables

These variables are automatically provided by md-pdf:

Variable	Type	Description	Example Value
`filepath`	String	Source markdown file path	**`"/path/to/document.md"`**
`default_author`	String	Author from configuration	**`"Your Name"`**
`language`	String	Document language	**`"en"`**, **`"de"`**, **`"fr"`**
`toc`	String	Table of contents flag	**`"true"`** or **`"false"`**



Variable	Type	Description	Example Value
`has_frontmatter`	String	Front matter presence indicator	**`"true"`** or **`"false"`**

4.3.2 Front Matter Variables

All YAML front matter fields become template variables.All fields are optional. Common fields are:

Field	Variable Name	Type	Example Value	Usage
`title`	**`title`**	String	**`"My Document"`**	Document title
`subtitle`	**`subtitle`**	String	**`"Guide"`**	Document subtitle
`author`	**`author`**	String	**`"John Doe"`**	Document author
`logo`	**`logo`**	String	**`"./logo.png"`**	Rectangular logo
`date`	**`date`**	String	**`"2024-01-22"`**	Publication date
`version`	**`version`**	String	**`"1.0.0"`**	Document version
`tags`	**`tags`**	Array	**`["docs", "guide"]`**	Document tags
`participants`	**`participants`**	Array	**`["Him", "Her"]`**	List of participants
`language`	**`language`**	String	**`"en"`**	Document language [**`en`**, **`de`**, **`fr`**]
`template`	**`template`**	String	**`"brutalist"`**	Template name [**`brutalist`**, **`playful`**, **`simple`**, **`none`**, **`your custom templatename`**]

4.3.3 Custom Fields

Any YAML field in front matter becomes a template variable:

```yaml
---
# Custom fields become template variables
client: "ACME Corporation"
contract_number: "CON-2024-001"
review_date: "2024-02-01"
budget: 50000
approved: true
reviewers:
    - "Alice Johnson"
    - "Bob Smith"
---
```

These become available as:

- **`sys.inputs.client`** ⟶ **`"ACME Corporation"`**
- **`sys.inputs.contract_number`** ⟶ **`"CON-2024-001"`**
- **`sys.inputs.budget`** ⟶ **`50000`**
- **`sys.inputs.approved`** ⟶ **`"true"`**
- **`sys.inputs.reviewers`** ⟶ **`["Alice Johnson", "Bob Smith"]`**

4.4 Using Variables in Templates

Templates use Typst syntax to access and manipulate variables:



4.4.1 Basic Variable Access

```
// Access system variables
#if sys.inputs.toc == "true" [
  #outline(title: "Table of Contents", depth: 3)
  #pagebreak()
]

// Use author with configuration fallback
#let doc_author = sys.inputs.at("author", default: sys.inputs.default_author)
```

4.4.2 Advanced Variable Processing

```
// Process arrays (tags, keywords, etc.)
#if "tags" in sys.inputs [
  *Tags:* #sys.inputs.tags.join(", ")
]
```

4.4.3 Variable Validation and Defaults

```
// Validate required fields
#let required_fields = ("title", "author", "version")
#for field in required_fields [
  #if field not in sys.inputs [
    #text(fill: red)[Error: Required field '#field' missing from front matter]
  ]
]

// Provide smart defaults
#let doc_title = sys.inputs.at("title", default: "Untitled Document")
#let doc_author = sys.inputs.at("author", default: sys.inputs.default_author)
#let doc_version = sys.inputs.at("version", default: "1.0")
```

4.5 Custom Template Development

4.5.1 Creating Custom Templates

1. **Navigate to templates directory**:

```
cd ~/.config/md-pdf/templates
```

2. **Create new template file**:

```
touch my-custom-template.typ
```

3. **Add template description** (recommended):

```
// My Custom Academic Template
// Professional template for academic papers and research documents
// Supports: citations, figures, academic formatting, multiple authors
// Created: 2024-01-22
// Last modified: 2024-01-22

// Your template code here...
```

4. **Use your template**:

```
md-pdf document.md -t my-custom-template
```

5 Front Matter Support

Front matter provides document metadata and per-document configuration through YAML headers at the beginning of Markdown files.

5.1 What is Front Matter?

Front matter is a YAML block at the very beginning of a Markdown file, enclosed by triple dashes (`---`). It contains metadata and configuration options that control how the document is processed and formatted.

5.2 Basic Front Matter

The simplest front matter includes core document information:

```yaml
---
title: "My Document"
author: "John Doe"
date: "2024-01-22"
tags:
    - tag1
    - tag2
---
# Document content starts here...
```

5.3 Supported Data Types

md-pdf handles all standard YAML data types with intelligent processing:

5.3.1 Strings

```yaml
# Simple strings
title: "Document Title"
description: "Single quoted string"

# Multi-line strings (preserve line breaks)
abstract: |
    This is a multi-line string
    that preserves line breaks.
    Perfect for abstracts, descriptions,
    or detailed explanations.

# Folded strings (join lines with spaces)
summary: >
    This is a folded string that joins
    multiple lines with spaces, creating
    a single paragraph of text.

# Escaped strings
special_chars: "Quotes: \"Hello\", Backslash: \\"
```

5.3.2 Numbers

```yaml
# Integers
version_number: 2
revision: 3
page_count: 42
budget: 50000
```



```yaml
# Floating point
price: 29.99
rating: 4.5
percentage: 95.7
```

5.3.3 Booleans

```yaml
toc: true
draft: false
confidential: true
approved: false
published: true
```

5.3.4 Arrays/Lists

```yaml
# Simple arrays
tags: ["markdown", "pdf", "documentation"]
languages: ["en", "es", "fr"]

# Multi-line arrays
keywords:
    - technical writing
    - automation tools
    - document processing
    - workflow optimization

# Mixed type arrays
scores: [95, 87.5, "excellent", true]

# Nested arrays
matrix:
    - [1, 2, 3]
    - [4, 5, 6]
    - [7, 8, 9]
```

5.3.5 Objects/Dictionaries

```yaml
# Simple objects
contact:
    name: "John Doe"
    email: "john@example.com"
    phone: "+1-555-0123"

# Nested objects
project_info:
    name: "Documentation Project"
    timeline:
        start: "2024-01-01"
        end: "2024-06-30"
    budget:
        allocated: 100000
        spent: 75000
    team:
        lead: "Jane Smith"
        members: ["Alice", "Bob", "Charlie"]
```

5.3.6 Dates and Times

```yaml
# ISO 8601 dates
date: "2024-01-22"
created: 2024-01-22T10:30:00Z
```




```
deadline: "2024-12-31T23:59:59Z"

# Natural language dates (processed as strings)
review_date: "January 31, 2024"
next_meeting: "first Monday of February"
```

5.4 Field Precedence and Smart Defaults

md-pdf uses intelligent precedence for configuration values:

1. **Front matter** (highest priority) - Values explicitly set in YAML
2. **Configuration file** - Defaults from your config file
3. **System defaults** (lowest priority) - Built-in fallbacks

5.5 Front Matter Example

```
---
title: "Machine Learning Applications in Document Processing"
subtitle: "A Comprehensive Survey and Future Directions"
author: "Dr. Sarah Johnson"
affiliation: "University of Technology"
department: "Computer Science"
date: "2024-01-22"
template: "academic"
toc: true
language: "en"
tags: ["machine learning", "document processing", "natural language processing", "computer
vision", "automation"]
abstract: |
    This paper presents a comprehensive survey of machine learning
    techniques applied to automated document processing tasks,
    including natural language processing, computer vision, and
    hybrid approaches. We analyze current methodologies, identify
    key challenges, and propose future research directions.
keywords:
    - machine learning
    - document processing
    - natural language processing
    - computer vision
    - automation
---
```

6 Working with Images

Standard Markdown image syntax renders an image at its natural size:

```
![md-pdf logo](resources/md-pdf.svg)
```



6.1 Resizing Images

Markdown has no syntax of its own for sizing images. Pandoc-style attributes such as `![](img.svg){.center width="40%"}` are **not** supported by the underlying **cmarker**/CommonMark parser and would appear as literal text.

Instead, use the HTML **<img>** tag, which md-pdf renders with support for **width**, **height**, and **align**:

- **width** / **height** — a percentage (**40%**, **40.0%**) or an absolute value in points (**120**).
- **align** — **left**, **center**, or **right**.

The same logo below is scaled down to 60% and then to 30% of the text width:

```
<img src="resources/md-pdf.svg" alt="md-pdf logo" width="60%">
<img src="resources/md-pdf.svg" alt="md-pdf logo" width="30%">
```





6.2 Aligning Images

Add **`align`** to place a resized image. The three examples below all use the **same `width="30%"`** so only the alignment changes — left, center, then right.

6.2.1 Left

```
<img src="resources/md-pdf.svg" alt="md-pdf logo" width="30%" align="left">
```





6.2.2 Center

```
<img src="resources/md-pdf.svg" alt="md-pdf logo" width="30%" align="center">
```



6.2.3 Right

```
<img src="resources/md-pdf.svg" alt="md-pdf logo" width="30%" align="right">
```



6.3 Tips

- Image paths that contain spaces must be wrapped in **<>** when using Markdown syntax: **`![alt](<my image.svg>)`**.
- For full control over placement you can inject raw Typst through an HTML comment:

```
<!--raw-typst #align(center)[#image("resources/md-pdf.svg", width: 40%)] -->
```



7 Usage Guide

7.1 Basic Commands

md-pdf provides a clean, intuitive command-line interface designed for efficiency and ease of use.

```
> md-pdf -h
Convert markdown files to PDF using typst with templating

Usage: md-pdf [OPTIONS] [INPUT]

Arguments:
  [INPUT]  Path to the input Markdown file

Options:
  -o, --output <OUTPUT>      Path to the output PDF file
  -t, --template <TEMPLATE>  Template to use for PDF generation [default: none]
  -w, --watch                Watch the input file for changes and rebuild automatically
      --check-links          Check all links in the markdown file and display warnings for
unreachable links
      --list-templates       List all available templates
      --create-config        Create default configuration file
      --show-config          Show configuration file locations and settings
      --open                 Open the generated PDF file after creation
      --refresh-templates    Refresh templates in the config directory with the latest
embedded versions
  -h, --help                 Print help
  -V, --version              Print version
```

7.1.1 Essential Operations

```
# Convert with all defaults (uses config settings)
md-pdf document.md

# Convert and open PDF automatically
md-pdf document.md --open

# Specify output filename
md-pdf document.md -o report.pdf

# Use specific template (overrides config default)
md-pdf document.md -t simple -o formatted-report.pdf

# Convert with template and auto-open
md-pdf document.md -t simple --open

# Watch file for changes (live preview mode)
md-pdf --watch document.md

# Watch and auto-open after each rebuild
md-pdf --watch document.md --open

# Watch with custom output and template
md-pdf --watch document.md -t simple -o live-preview.pdf

# Watch with custom output, custom template, auto-open, and link checking
md-pdf --check-links --watch document.md -t simple --open -o live-preview.pdf
```



7.1.2 Information and Configuration Commands

```
# List all available templates with descriptions
md-pdf --list-templates

# Show current configuration and all paths
md-pdf --show-config

# Create default configuration file (if not exists)
md-pdf --create-config

# Show help information
md-pdf --help

# Show version information
md-pdf --version
```

7.1.3 Link Checking and Validation

md-pdf includes built-in link checking functionality to validate external links in your markdown documents before PDF conversion.

```
# Check all external links in document
md-pdf --check-links document.md
```

7.1.3.1 Sample Output

```
🔍 Checking 5 links in 'research-paper.md'...
✅ https://github.com/typst/typst
✅ https://www.rust-lang.org
❌ https://broken-example.invalid - Error: dns error: failed to lookup address information
✅ https://docs.rs
✅ https://wikipedia.org

📊 Link check summary:
  ✅ Successful: 4
  ❌ Failed: 1
⚠  Some links are not reachable. Consider reviewing them.

Using template: simple
✓ Successfully converted 'research-paper.md' to 'research-paper.pdf'
```